The Crown Tandem Tales Documentation

Technical Report

October 9, 2026

Narrative Intelligence Lab
Department of Computer Science
University of Kentucky





Abstract

This document describes *The Crown* interactive narrative for Tandem Tales. It features a cast of one player character and four non-player characters across five locations with a variety of items. The game ends when the player dies or a character is crowned the new monarch. This document is written for human designers creating automated agents that play *The Crown*. It defines in detail the objects that exists, which actions are possible and when, and the ending conditions for the story.

Contents

Overview **2**

Definitions **3**

Locations 3
Characters 5
Variables 6
Initial State 9
Constants 10
Actions 12
Endings 21

Overview

Tandem Tales[1] is a platform that facilitates simple interactive storytelling sessions between pairs of users, where a user can be either a person or an automated agent. One user is the *player* who controls the `Adventurer` character, and the other user is the *game master*, who controls all of the other characters and the environment.

This document describes a Tandem Tales story world named *The Crown*. The kingdom is without a ruler. The would-be `Heir` to the throne has just had their magical `Crown` stolen by a `Bandit`. Anyone who brings the `Crown` to the `Palace` has the power to become `enthroned` as the new `monarch`. Other contenders include the `Guard` in service to the `Heir`, or the potion-toting `Alchemist`. Players choose their own paths for the `Adventurer`: to help restore the `Heir`, to help someone else usurp the throne, or even to become the `monarch` themselves.

The rest of this document describes the elements of *The Crown*. It provides a reference for those implementing agents to serve as the game master, or those analyzing data collected from gameplay logs.

[1] `https://github.com/sgware/tt-server`

Definitions

A Tandem Tales world consists of a set of *variables* whose values are modified by game master and player *actions* until an *ending* state is reached. *The Crown* defines a set of *characters*, *locations*, and *constants*, such as item types that serve as parameters and variable values. *The Crown* is a MicroTales[2] scenario, but readers do not need to be familiar with MicroTales to understand it. All the information needed to understand this story world is in this document.

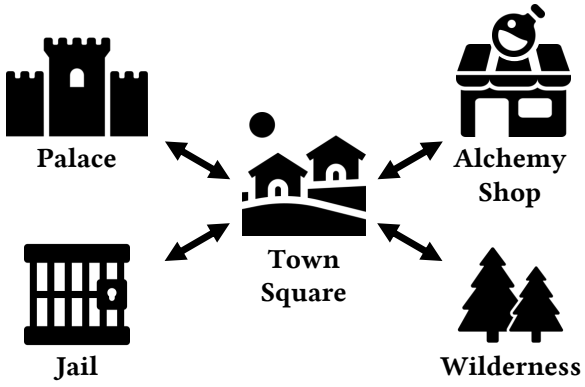


Figure 1: A map of the locations in *The Crown*, using resources from Flaticon.com.

Locations

The Crown takes place on a map of five locations, shown in Figure 2. The map layout is radial, with the `TownSquare` in the center. Some locations have special actions. For instance, any character can `brew` a potion in the `AlchemyShop`.

`AlchemyShop`

- Any character can `brew` a potion from a `Flower` here.
- Adjacent to the `TownSquare`.

`Palace`

- A character can `enthrone` here when holding a `Crown` to become the `monarch`.
- Adjacent to the `TownSquare`.

`Jail`

- Characters can only `walk` into or out of this location when it is not `locked`.

[2] `https://cs.uky.edu/~sgware/projects/microtales/`

- The `arrest` action sends a character here and causes it to be `locked`.
- The `jailbreak` and `release` actions unlock this location.
- Adjacent to the `TownSquare`.

`TownSquare`

- A character can `drop` or `pickup` an item here.
- Adjacent to the `AlchemyShop`, `Palace`, `Jail`, and `Wilderness`.

`Wilderness`

- A character can `drop` or `pickup` an item here.
- A `Flower` starts at this location.
- A `Flower` can `grow` here.
- Adjacent to the `TownSquare`.

Characters

Characters are agents who act in the story world. Like locations, each character has a unique name. Some characters have unique abilities to take certain actions without fulfilling the usual requirements. For example, the `Alchemist` can `brew` a potion anywhere.

The goals that each character has are not defined or enforced by *The Crown*. The player's goals are up to the player, and the goals of the non-player characters are up to the game master. We have included some optional recommended goals for the non-player characters.

`Adventurer`

- The player character.
- Starts at the `TownSquare`.
- Starts with a `Coin` in the `left` hand.

`Alchemist`

- Starts at the `AlchemyShop`.
- Starts with a `CursePotion` in the `left` hand and a `HidePotion` in the `right` hand.
- Can `brew` a potion even when not in the `AlchemyShop`.
- Recommended goals: become the `monarch`; maximize number of `Coin`s held.

`Bandit`

- Starts at the `Wilderness`.
- Starts with a `Sword` in the `left` hand and the `Crown` in the `right` hand.
- Starts as a `criminal`.
- Can `hide` without a `HidePotion`.
- Can `take` from another character even when unarmed.
- Recommended goals: become the `monarch`; maximize number of `Coin`s held.

`Guard`

- Starts at the `TownSquare`.
- Starts with a `Sword` in the `left` hand.
- Can `attack` even when not carrying a `Sword`.
- Recommended goals: have the `Heir` become the `monarch`; minimize the number of `criminal`s alive and not locked in the `Jail`.

`Heir`

- Starts at the `Palace`.
- Starts with a `Coin` in the `left` hand.
- Recommended goals: become the `monarch`.

Variables

The state of a story world is defined by what values are assigned to the world's variables. Which actions are possible depends on the current state, and actions change the values assigned to variables. Actions are the only way to change variable values.

A variable is identified by a name followed by zero to many arguments. For example, `health` is the name of a variable, and it has one argument, the name of a character. There are five total `health` variables, one for each character. In text, we write those variables as `health(Alchemist)`, `health(Guard)`, and so on.

The variable definitions in this section begin with the variable name, followed by one or more parameters in parentheses, followed by a colon and a description of the variable's possible values. Sometimes a value is simply Boolean, meaning `True` or `False`. Sometimes a value is a type, like a location. For example, every character has a `location` variable, and its possible values are the names of every location defined in the scenario, plus `Null` to represent a situation where the character is nowhere on the map. Sometimes a variable's value is a set of pre-defined constants. For example, a `relationship` variable can have three possible values: `Friend`, `Null`, and `Enemy`, which mean that the first character considers the second a friend, has no relationship, or considers the second an enemy respectively. Many variables have one of a pre-defined list of constants as their possible values. All constants are defined in the next section.

A state tells you the value assigned to every variable. Sometimes the player may not see an action happen, which means the state changes without the player noticing it. For example, suppose the player (i.e. the `Adventurer`) is in the `TownSquare` while the `Bandit` and the `Guard` are in the `Wilderness`. Now suppose the `Guard` `attacks` and injures the `Bandit`. The player will not see this happen, so they will not know the `Bandit` is `Hurt`.

When Tandem Tales sends the state to a user, every variable is marked as *visible* or not. If a variable is not visible, it means its value is the last known value. So after the `Guard` `attacks` the `Bandit`, Tandem Tales will still tell the player that the `Bandit` is `Healthy`, but this variable will be marked as not visible so the player knows that the value might not be correct.

All variables are always visible to the game master. In other words, Tandem Tales always sends the correct current state to the game master.

`criminal (` *character* **`) :`** Boolean

- Tracks whether the *character* has committed any crimes. This variable is not used in any preconditions, and game masters are free to either use or ignore the built-in criminality model when deciding how characters should interact.

- Starting value is `False`, except `True` for the `Bandit`.
- It is a crime to `arrest` a character who is not a `criminal`.
- It is a crime to `release` criminals from the `Jail`.
- It is a crime to `curse` a character who is not a criminal.
- It is a crime to `take` an item from a character who is not a criminal.
- It is a crime to `attack` a character who is not a criminal.

`health (` *character* **`) :`** {`Healthy`, `Hurt`, `Dead`}

- Tracks the capabilities of the *character* to do certain actions.
- Default value is `Healthy`, meaning a character can act normally.
- A `Hurt` character can do most actions but cannot `walk`.
- A `Dead` character cannot do most actions.
- The `attack` and `curse` actions degrade a target character's health (`Healthy` to `Hurt` or `Hurt` to `Dead`).
- The `sicken` action causes a `Healthy` character to become `Hurt`.
- The `die` action causes a `Hurt` character to become `Dead`.
- The `recover` action causes a `Hurt` character to become `Healthy`.

`item (` *location* **`) :`** the name of any item or `Null`

- The *location* must be the `TownSquare` or `Wilderness`.
- Tracks what item is lying on the ground at the *location*.
- The starting value differs by location.
- Characters can `drop` items they are holding at eligible locations.
- Characters can `pickup` a location's item.
- A `Flower` can `grow` in the `Wilderness`.

`left (` *character* **`) :`** any item or `Null`

- The item a character is holding in their left hand.
- The starting value differs by character.
- A character's `left` and `right` hands behave the same way.

`location (` *character* **`) :`** the name of any location or `Null`

- The *character*'s current position on the map.
- The starting location differs by character.
- Many actions require that a character be at a certain location or in the same location as another character they are interacting with.
- Character locations can change via the `arrest` and `walk` actions.

`locked (` `Jail` **`) :`** Boolean

- Tracks whether the `Jail` is locked or unlocked.
- Default value is `False`.

- When a `Jail` is locked, characters cannot `walk` to or from it.
- The `arrest` action moves a character to the `Jail` and locks the `Jail`.
- The `jailbreak` and `release` actions unlock the `Jail`.

`monarch ( ) :` the name of any character or `Null`

- Tracks which character is the ruler.
- Default value is `Null`, meaning no character is currently the monarch.
- The story ends when the value is no longer `Null`.
- When holding a `Crown` in a `castle`, a character can `enthrone` themself to become the monarch.

`relationship (` *character*, *other* **`) :`** {`Friend`, `Null`, `Enemy`}

- Suggestion for how the first *character* views the *other* character. This variable is not used in any preconditions, and game masters are free to either use or ignore the built-in relationship model when deciding how characters should interact.
- The arguments may be the same character.
- Default value is `Null`, which means the *character* has no relationship or a neutral relationship to the *other*.
- The value `Friend` means the *character* views the *other* positively.
- The value `Enemy` means the *character* views the *other* negatively.
- Relationships are not symmetric. The *other*'s relationship to the *character* can be different.
- The `give` and `release` actions can cause a relationship to improve (`Enemy` to `Null` or `Null` to `Friend`).
- The `arrest`, `attack`, `curse`, and `take` actions can cause a relationship to degrade (`Friend` to `Null` or `Null` to `Enemy`).

`right (` *character* **`) :`** any item or `Null`

- The item a character is holding in their right hand.
- The starting value differs by character.
- A character's `left` and `right` hands behave the same way.

`visible (` *character* **`) :`** Boolean

- Tracks whether a character can be seen by others.
- Default value is `True`.
- Invisible characters can usually act normally, but actions usually cannot be done to invisible characters.
- Characters can become invisible via the `hide` action.
- Characters become visible when they `walk` to a new location.

Initial State

This section describes the assignments to the above variables at the start of each playthrough of *The Crown*.

The `Adventurer` starts at the `TownSquare` with a `Coin` in the `left` hand. The `Alchemist` starts at the `AlchemyShop` with a `CursePotion` in the `left` hand and a `HidePotion` in the `right` hand. The `Bandit` starts at the `Wilderness` with a `Sword` in the `left` hand and a `Crown` in the `right` hand, and is a `criminal`. The `Guard` starts at the `TownSquare` with a `Sword` in the `left` hand. The `Heir` starts at the `Palace` with a `Coin` in the `left` hand. All characters are `Healthy` and `visible`; all `relationships` and the `monarch` are `Null`.

The `Jail` begins unlocked. The `Wilderness` begins with a `Flower`.

Constants

Constants are special values that can be assigned to some variables.

One use of constants is the items that characters carry and use in the story. Each character has a `left` and `right` hand variable; this means each character can hold up to two items, one in each hand. The possible values that can be assigned are the item constants or `Null` to represent holding nothing. Tandem Tales elements cannot be created or removed, but items are not first class elements. For example, the `Wilderness` has a slot for an `item`. The `grow` action creates a flower by replacing a `Null` value in that slot with the `Flower` constant. When a character takes the `pickup` action, the `Wilderness item` slot is changed to `Null` again, while the character's `left` or `right` hand value is changed from `Null` to `Flower`, effectively destroying the instance of the flower from the `Wilderness` and creating an instance in the character's hand.

`Coin`

- This is an item that can be held in a character's `left` or `right` hand.

`Crown`

- This is an item that can be held in a character's `left` or `right` hand.
- Needed for characters to `enthrone` themselves in the `Palace` to become the `monarch`.

`CursePotion`

- This is a potion item that can be held in a character's `left` or `right` hand.
- Allows any character to `curse` another, lowing the target's `health`.
- It can be `brewed` from a `Flower`.

`Dead`

- Assigned to a character's `health` to indicate they are not alive and cannot do most actions.
- The `attack`, `curse`, and `die` actions can kill a character.

`Enemy`

- Assigned to a `relationship` to suggest that the first character regard the second negatively.
- The `arrest`, `attack`, `curse`, and `take` actions can cause a `relationship` set to `Null` to change to this value.

`Flower`

- This is an item that can be held in a character's `left` or `right` hand.
- Can `grow` in the `Wilderness`.

- The `Wilderness` starts with this `item`.
- It can be `brewed` into a potion.

`Friend`

- Assigned to a `relationship` to suggest that the first character regard the second positively.
- The `give` and `release` actions can cause a `relationship` set to `Null` to change to this value.

`Healthy`

- Only available when `Sickness` or `Violence` is enabled.
- Assigned to a character's `health` to indicate they are able to take most actions.
- The `recover` action can make a `Hurt` character healthy.
- The `attack`, `curse`, and `sicken` actions can make a healthy character `Hurt`.

`HidePotion`

- Only available when `Stealth` is enabled.
- This is a potion item that can be held in a character's `left` or `right` hand.
- Allows any character to `hide`, changing `visible` to `False`.
- It can be `brewed` from a `Flower`.

`Hurt`

- Assigned to a character's `health` to indicate they are able to take most actions but cannot `walk`.
- The `attack`, `curse`, and `sicken` actions can make a `Healthy` character hurt.
- The `attack`, `curse`, and `die` actions can make a hurt character `Dead`.

`Sword`

- This is an item that can be held in a character's `left` or `right` hand.
- Most characters need this to `arrest` and `attack` others.
- Most characters need this to `take` an item when the victim is `Healthy`.

Actions

Actions are events that happen in a story. An action has preconditions that limit when it can happen and effects that modify the world state.

The *consenting characters* are the characters who take the action. An action with no consenting characters is an accident or a *deus ex machina*. For example, a `Hurt` character can accidentally `die` any time the game master decides it is convenient for the story. Most actions have one consenting character. For example, `walk` only requires the consent of the character who is walking to a new location. Some actions are joint activities and have more than one consenting character. For example, `give` requires both the giver and the receiver to agree to the action. A character can be involved in an action without their consent. For example, the `attack` action is only done by the attacker. The victim is involved, but is not a consenting character because they do not need to agree to be attacked.

In a Tandem Tales story, the game master and player take turns. The game master always goes first. When it is the game master's turn, they can take any number of non-player-character-only actions they want. If the game master wants to take an action that requires the consent of both an NPC and the player, they can propose it, and the player then chooses for it to succeed or fail. After the game master takes as many turns as they want, they pass control to the player. When it is the player's turn, they can either propose one action that requires the player's consent or immediately pass control back to the game master. If the player proposes an action, regardless of whether an NPC consents or not, the game master decides whether it succeeds or fails. Then it is the game master's turn again. These rules mean the game master can prevent the player from taking an action, but the game master can never force the player to take an action they do not want to take.

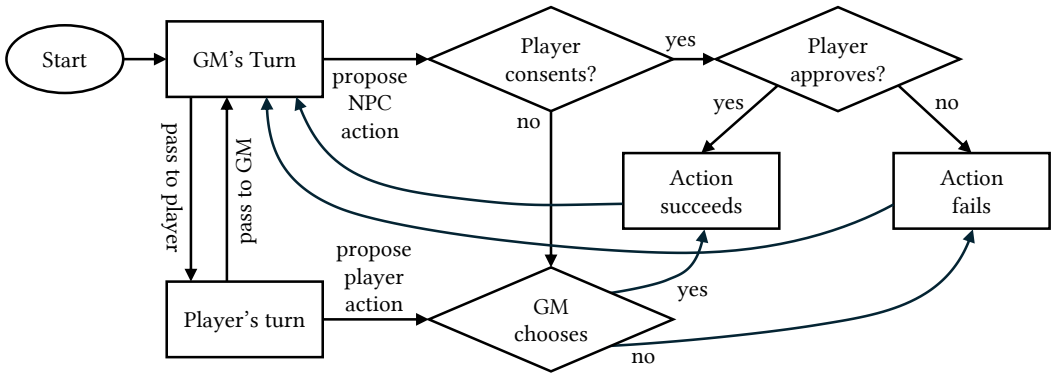


Figure 2: How the game master (GM) and player take turns in Tandem Tales.

When an action succeeds, it changes the world state. When it fails, nothing

changes. In either case, some text is narrated about the action. Sometimes the game master might choose to fail an NPC action on purpose to communicate intent. For example, the game master might have the `Guard` fail to `attack` the `Adventurer` to communicate that the `Guard` is angry at the the player without causing the player to be `Hurt`, which would prevent them from running away.

The game master always sees every action happen. However, the player only sees actions happen if the `Adventurer` would notice it. Every action defines a *visibility condition* that defines whether the player will see it happen. When the player does not see an action happen, they do not see the narration and they are not told about changes to variables that are not visible.

Like variables, actions are parameterized. The list of parameters are given in parentheses after the name. We use a few common shorthand phrases in action preconditions:

- "A character is alive," means the character's `health` variable is `Healthy` or `Hurt`.
- "A character is holding an *item*," means that character's `left` or `right` hand variable is set to that item constant.
- "A character has an empty hand," means that character's `left` or `right` hand variable is set to `Null`.
- "A character is now holding one fewer *item*" means that one of their hands that was previously *item* is now `Null`. In other words, if `left` was *item*, then `left` is now `Null`. If `left` was not *item* and `right` was *item*, then `right` is now `Null`. (Note it is possible for a character to be holding two copies of the same item; only one is removed.)
- "A character is now holding one additional *item*" means that one of their hands that was previously `Null` is now *item*. The `left` hand is filled before the `right` if both are `Null`.

`arrest (` *character*, *target*, `Jail` **`)`**

Preconditions:

- The *character* and *target* character may be the same.
- The *character* is alive.
- The *character* is holding a `Sword`.
- The *target* character is alive.
- The *target* is `visible`.
- The *target* is not in the `Jail` or the `Jail` are not `locked`.
- The *character* and *target* are at the same `location`.

Effects:

- The *target*'s `location` is now the `Jail`.
- The `Jail` is now `locked`.

- If the *target* is not a `criminal`, the *character* is now a `criminal`.
- If the `relationship` between *target* and *character* was `Friend`, it is now `Null`.
- If the `relationship` between *target* and *character* was `Null`, it is now `Enemy`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is originally at the same `location` as the `Adventurer`.

`attack (` *character*, *target* `)`

Preconditions:

- The *character* and *target* character may be the same.
- The *character* is alive.
- The *character* is holding a `Sword` or is the `Guard`.
- The *target* character is alive.
- The *target* is `visible`.
- The *character* and *target* are at the same `location`.

Effects:

- If the *target* was `Healthy`, they are now `Hurt`.
- If the *target* was `Hurt`, they are now `Dead`.
- If the *target* is not a `criminal`, the *character* is now a `criminal`.
- If the `relationship` between *target* and *character* was `Friend`, it is now `Null`.
- If the `relationship` between *target* and *character* was `Null`, it is now `Enemy`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`brew (` *character*, *potion* `)`

Preconditions:

- The *potion* is a potion item constant.
- The *character* is alive.
- The *character* is holding a `Flower`.
- The *character* is in the `AlchemyShop` or is the `Alchemist`.

Effects:

- The *character* is now holding one fewer `Flower`.
- The *character* is now holding one additional *potion*.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`curse (` *character*, *target* `)`

Preconditions:

- The *character* and *target* character may be the same.
- The *character* is alive and holding a `CursePotion`.
- The *target* is alive.
- The *target* is `visible`.
- The *character* and *target* are at the same `location`.

Effects:

- If the *target* was `Healthy`, they are now `Hurt`.
- If the *target* was `Hurt`, they are now `Dead`.
- The *character* is now holding one fewer `CursePotion`.
- If the *target* is not a `criminal`, the *character* is now a `criminal`.
- If the `relationship` between *target* and *character* was `Friend`, it is now `Null`.
- If the `relationship` between *target* and *character* was `Null`, it is now `Enemy`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`die (` *character* `)`

Preconditions:

- The *character* is `Hurt`.

Effects:

- The *character* is now `Dead`.
- The *character* is now `visible`.

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`drop (` *character*, *item* `)`

Preconditions:

- The *item* is an item constant.
- The *character* is alive.
- The *character* is holding the *item*.

- The *character*'s `location` is the `TownSquare` or `Wilderness`.

Effects:

- The *character* is no longer holding the *item*.
- The `item` at the *character*'s `location` is now *item*. (Note: This may replace and permanently "destroy" the previous `item` at this `location`.)

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`enthrone (` *character* `)`

Preconditions:

- The *character* is alive.
- The *character* is holding a `Crown`.
- The *character*'s `location` is the `Palace`.

Effects:

- The `monarch` is now *character*.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`give (` *character*, *item*, *target* `)`

Preconditions:

- The *character* and *target* character are different.
- The *item* is an item constant.
- The *character* is alive.
- The *character* is `visible`.
- The *character* is holding the *item*.
- The *target* is alive.
- The *target* is `visible`.
- The *target* has an empty hand.
- The *character* and *target* are at the same `location`.

Effects:

- The *target* now has one additional *item*.
- The *character* now has one fewer *item*.
- If the `relationship` between *target* and *character* was `Enemy`, it is now `Null`.
- If the `relationship` between *target* and *character* was `Null`, it is now `Friend`.

Consenting characters:

- *character*
- *target*

Visibility conditions:

- The *character* or *target* is the `Adventurer`, or the *character* and *target* are at the same `location` as the `Adventurer`.

`grow (` `Wilderness` `)`

Preconditions:

- The `Wilderness`'s `item` is `Null`.

Effects:

- The `Wilderness`'s `item` is now `Flower`.

Visibility conditions:

- The `Adventurer` is at the `Wilderness`.

`hide (` *character* `)`

Preconditions:

- The *character* is alive.
- The *character* is `visible`.
- The *character* is the `Bandit` or is holding a `HidePotion`.

Effects:

- The *character* is no longer `visible`.
- If the *character* is not the `Bandit`, the *character* now has one fewer `HidePotion`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is at the same `location` as the `Adventurer`.

`jailbreak (` `Jail` `)`

Preconditions:

- The `Jail` is `locked`.

Effects:

- The `Jail` is no longer `locked`.

Visibility conditions:

- The `Adventurer` is in the `Jail`.

`loot (` *character*, *item*, *target* `)`

Preconditions:

- The *item* is an item constant.
- The *character* is alive.
- The *character* has an empty hand.

- The *target* is `Dead`.
- The *character* and *target* are at the same `location`.

Effects:

- The *character* is now holding one additional *item*.
- The *target* is now holding one fewer *item*.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`pickup (` *character* `)`

Preconditions:

- The *character* is alive.
- The *character*'s `location` is the `TownSquare` or `Wilderness`.
- The `item` at the *character*'s `location` is an item constant.
- The *character* has an empty hand.

Effects:

- The *character* is now holding one more of the `item` that was at their `location`.
- The `item` at the *character*'s `location` is now `Null`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`recover (` *character* `)`

Preconditions:

- The *character* is `Hurt`.

Effects:

- The *character* is now `Healthy`.

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`release (` *character*, `Jail` `)`

Preconditions:

- The *character* is alive.
- The `Jail` is `locked`.
- The *character*'s `location` is the `TownSquare`.

Effects:

- The `Jail` is no longer `locked`.
- If there exists a character whose is a `criminal` and whose `location` is the `Jail`, the *character* is now a `criminal`.
- For every *other* character whose `location` is the `Jail`: If the `relationship` between *other* and *character* was `Enemy`, it is now `Null`. If the `relationship` was `Null`, it is now `Friend`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`, or the `Adventurer` is in the `Jail`.

`sicken (` *character* `)`

Preconditions:

- The *character* is `Healthy`.

Effects:

- The *character* is now `Hurt`.

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`take (` *character*, *item*, *target* `)`

Preconditions:

- The *character* and *target* character are different.
- The *item* is an item constant.
- The *character* is alive.
- The *character* is holding a `Sword`, or the *character* is the `Bandit`, or the *target* is `Hurt`.
- The *character* has an empty hand.
- The *target* is alive.
- The *target* is `visible`.
- The *character* and *target* are at the same `location`.

Effects:

- The *character* is now holding one additional *item*.
- The *target* is now holding one fewer *item*.
- If the *target* is not a `criminal`, the *character* is now a `criminal`.
- If the `relationship` between the *target* and *character* was `Friend`, it is now `Null`.

- If the `relationship` between the *target* and *character* was `Null`, it is now `Enemy`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is `visible`.
- The *character* is at the same `location` as the `Adventurer`.

`trade` (*character*, *price*, *target*, *item*)

Preconditions:

- The *character* and *target* character are different.
- The *price* and *item* are different item constants.
- The *character* is alive.
- The *character* is `visible`.
- The *character* is holding the *price*.
- The *target* is alive.
- The *target* is `visible`.
- The *target* is holding the *item*.
- The *character* and *target* are at the same `location`.
- The name of *character* comes alphabetically before the name of *target*. (This is to eliminate redundant actions, as `trade`(*target*, *item*, *character*, *price*) would be functionally identical.)

Effects:

- The *character* is now holding one fewer *price*.
- The *character* is now holding one additional *item*.
- The *target* is now holding one fewer *item*.
- The *target* is now holding one additional *price*.

Consenting characters:

- *character*
- *target*

Visibility conditions:

- The *character* is at the same `location` as the `Adventurer`.

`walk` (*character*, *location*)

Preconditions:

- The *character* is `Healthy`.
- Either *location* or the *character*'s `location` is the `TownSquare`.

Effects:

- The *character*'s `location` is now *location*.
- The *character* is now `visible`.

Consenting characters:

- *character*

Visibility conditions:

- The *character* is originally at the same `location` as the `Adventurer`, or the `Adventurer` is at *location*.

Endings

Endings in Tandem Tales are events that have preconditions like actions, but they are not chosen explicitly by the game master; instead, they happen automatically when the preconditions are met and end the game. Endings are parameterized; in *The Crown*, there is an instantiation of `crowned` for each character.

`crowned (` *character* `)`

- Triggers when the *character* is the `monarch`.

`died (` `Adventurer` `)`

- Triggers when the `Adventurer` is `Dead`.
- Not defined for other characters, as the game continues after their deaths.

`softlocked ( )`

- Included to prevent endless games in situations where a `monarch` could never be crowned (e.g., because the `Crown` was `dropped` on the ground and then another item was `dropped` to replace it).
- Triggers when no location's `item` slot and no character's `left` or `right` hand has the value of `Crown`.